496 Software Project:
Plant Watch

Brenden Bokino and Nishant Gurung

2026-04-01

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr. Maren Blohm
- Client title: Professor of Biology
- Client email address: mblohm@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: Loyola professor, met in person.

2 Project Description

2.1 Overview

Our project will be to build a mobile application that can be used to manage plant life. The app will allow users to scan plants and assess their condition, suggesting potential watering schedules and nutritional supplements to help their growth. The app will also allow users to manage greenhouses and interact with devices that might be used in them. Some of the devices Dr. Blohm is interested in are devices that can monitor moisture levels in soil, weather conditions, and automatic sprinklers.

These devices would be in the greenhouse and the garden beds we have around the campus. The information provided by our software will help them make better decisions about caring for these plants. It will also save time for the department, requiring them to check on the plants less often. With the Biology Department relocating, having to travel to the greenhouses less would be ideal.

2.2 Key Features

1. Scanning a plant to identify it and possibly note nutrient deficiencies and watering needs.
2. Individualized profiles for users that lets them manage their greenhouses and their devices.
3. Connect with and pull information from devices in the greenhouses such as weather monitors and moisture monitors.
4. Allow users to activate devices remotely or configure devices to activate in certain conditions (ex. Automatically activate sprinklers when moisture in the soil is below a certain threshold)

2.3 Why this Project is Interesting

This project bridges the two fields of Computer Science and Biology by showing how technology can contribute directly to environmental conservation efforts. This capstone gives us a chance to make a real, measurable impact in preserving the biodiversity of Loyola's campus. Additionally, it will build a model that can be adapted by other institutions seeking to monitor and protect the nature around them. The skills that will be learned are interesting as well. Computer vision is something we both do not have worked with and learning to connect different devices together will be a good experience.

2.4 Areas of CS required

1. Mobile app development
2. Computer Vision
3. Networking
4. Database

2.5 Potential Concerns and Questions

1. We are unsure of the difficulty of developing with computer vision. There are likely resources out there on it, but we are concerned with how accurate it can get.
2. We are unfamiliar with the devices used when monitoring plants and how programmable they will be.

2.6 Summary of Efforts to Find a Project

We've been discussing about this project with Dr. Blohm since the last semester. She has guided us towards the right direction and answered our questions about the greenhouse, plant health, and the requirements of the app.

2.7 Comparison to Draft

The project was initially proposed by me (Nishant) during the drafting phase. While the core concept of building an app that uses image recognition to inform the users about its water cycle, we've changed the proposal to incorporate more fields of Computer Science. The initial draft was largely centered on Computer Vision, particularly image recognition for species identification. But the present proposal is not limited to a single subfield; instead, it is divided into several subfields of Computer Science that include:

- Computer Vision for automated species detection
- Database design and management for storing and organizing plant observations, weather patterns, and health assessment data to enable long-term tracking of garden performance.
- Networking for real-time data synchronization with the weather monitoring devices and multi-user access, which will send immediate alerts on temperature drops, and soil moisture to prevent crop loss.
- Mobile App Development for creating an intuitive, user-friendly interface.

3 Requirements

3.1 Non-Functional Requirements

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR1	App Launch Time	Performance	The application must launch and display the home screen in under 3 seconds.
NFR2	Plant Identification Accuracy	Reliability	The plant identification feature must have a minimum accuracy rate of 85% across common plant species.
NFR3	Scan Process Efficiency	Usability	The scan-to-identify process must require a maximum of 3 user taps to complete.
NFR4	Outdoor Accessibility	Usability	The application must use accessible font sizes and appropriate color contrast for outdoor visibility and readability.
NFR5	Visual Design Consistency	Usability	The application must maintain a consistent color scheme with green as the primary color and white as the secondary color.
NFR6	Platform Compatibility	Compatibility	The application must support iOS and Android.
NFR7	Battery Efficiency	Performance	The application must consume less than 5% of device battery per hour during active use.
NFR8	Alert Delivery Speed	Performance	Emergency alerts for critical conditions (temperature drops, low moisture) must be delivered to users within 30 seconds of detection.
NFR9	Password Encryption	Security	Passwords should be encrypted before being stored in the database.

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2 presents the functional requirements for this software project.

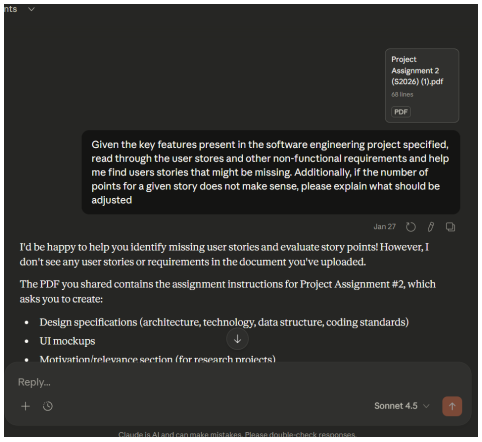
Table 2: Functional requirements as User Stories.

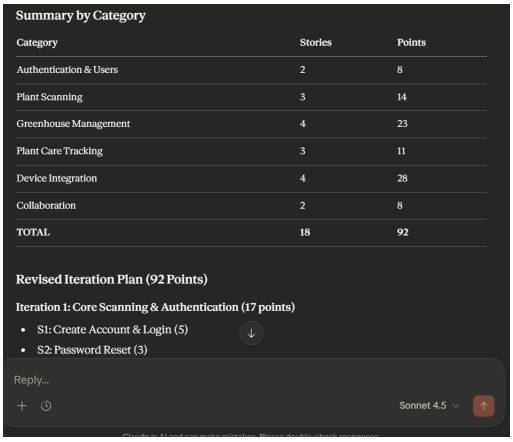
ID	Story Title	Points	Description
S1	Scan Plants for Detailed Information	8	As a user I want to scan a plant with my camera and get information about what plant it is, what watering schedules would be best, and what nutritional deficiencies it might have so I can learn more about plants and how to care for them.
S2	Password Reset	3	As a user I want to be able to reset my password if I forget it so that I can configure a new password and login.
S3	Add Plants to Greenhouse by Scan	2	As a user I want to add a plant that I have scanned to my greenhouse profile so that I do not manually have to input the information I learned from the scan.
S4	View Plant Scan History	3	As a user I want to view the history of plants I have scanned so I don't have to rescan the same plant repeatedly.
S5	Create Account	3	As a user, I want to be able to create an account so I can have my own personal profile.
S6	Greenhouse Profile Management	8	As a user, I want to create, view, edit, and delete a greenhouse profile so I can manage my plants and plant monitoring devices.

ID	Story Title	Points	Description
S7	Greenhouse Profile Dashboard	5	As a user, I want to view a dashboard of my greenhouse profile with basic information on its plants' and devices' conditions so I can quickly analyze its state.
S8	Sync AcuRite Weather Sensor	5	As a user I want to be able to view information from my AcuRite weather sensor from the app so I know the conditions my plants are in.
S9	Add Plants to Greenhouse	8	As a user I want to be able to create, view, edit, and delete plants in my greenhouse, so that I can keep inventory of plants.
S10	Greenhouse Notifications	5	As a user I want to receive notifications from greenhouses that I am in about conditions that may negatively affect plants and customize what those conditions may be so that I can know when I need to go in to care for them.
S11	Adding Users to Greenhouse	2	As a greenhouse manager I want to add other users to my greenhouse so they can view its information and manage it.
S12	Scheduled Sprinkler Activation	5	As a user I want to sync a sprinkler with the app and configure schedules for automatic sprinkler activation so that I can water plants without having to go to the greenhouse. (Still in discussion with client about these devices)
S13	Greenhouse User Device Use Permissions	2	As a greenhouse manager I want to be able to restrict the usage of devices in my greenhouse to certain users I've invited so I can safely collaborate with other users.
S14	Greenhouse User Plant Edit Permissions	2	As a greenhouse manager I want to restrict the ability to edit plant information to certain users so I can ensure the plant information is coming from a trusted source.
S15	Greenhouse Activity Log	3	As a greenhouse manager I want to be able to view a history of actions taken and edits made to my greenhouse so I can be sure the right users are doing the right things.
S16	Greenhouse Plant Info	2	As a user I want to view detailed information about a plant that I've added to my greenhouse so I can tailor its care and learn more about it.
S17	Sync Soil Moisture Sensor	5	As a user I want to sync information from my soil moisture sensor to the app so that I can view it along with other greenhouse information and make decisions about its care. (Still in discussion with client about these devices)
S18	View Historical Conditions	5	As a user I want to have a graph view of various collected historical information from my greenhouses devices such as humidity, temperature, etc. so I can make informed decisions concerning my plants.
S19	Login/Logout	1	As a user, I want to be able to login and logout of an account so that I can access the app features that require an account.
	Total:	77	

4 AI Usage

4.1 Conversation





Summary by Category

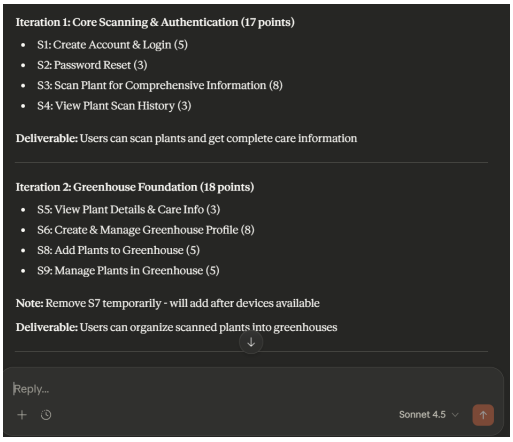
Category	Stories	Points
Authentication & Users	2	8
Plant Scanning	3	14
Greenhouse Management	4	23
Plant Care Tracking	3	11
Device Integration	4	28
Collaboration	2	8
TOTAL	**18**	**92**

Revised Iteration Plan (92 Points)

Iteration 1: Core Scanning & Authentication (17 points)

- S1: Create Account & Login (5)
- S2: Password Reset (3)

Reply...
Sonnet 4.5



Iteration 1: Core Scanning & Authentication (17 points)

- S1: Create Account & Login (5)
- S2: Password Reset (3)
- S3: Scan Plant for Comprehensive Information (8)
- S4: View Plant Scan History (3)

Deliverable: Users can scan plants and get complete care information

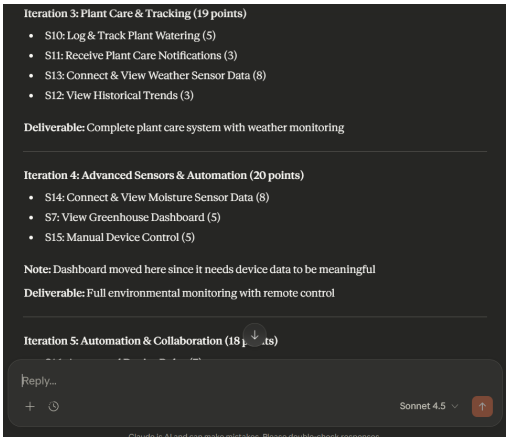
Iteration 2: Greenhouse Foundation (18 points)

- S5: View Plant Details & Care Info (3)
- S6: Create & Manage Greenhouse Profile (8)
- S8: Add Plants to Greenhouse (5)
- S9: Manage Plants in Greenhouse (5)

Note: Remove S7 temporarily - will add after devices available

Deliverable: Users can organize scanned plants into greenhouses

Reply...
Sonnet 4.5



Iteration 3: Plant Care & Tracking (19 points)

- S10: Log & Track Plant Watering (5)
- S11: Receive Plant Care Notifications (3)
- S13: Connect & View Weather Sensor Data (8)
- S12: View Historical Trends (3)

Deliverable: Complete plant care system with weather monitoring

Iteration 4: Advanced Sensors & Automation (20 points)

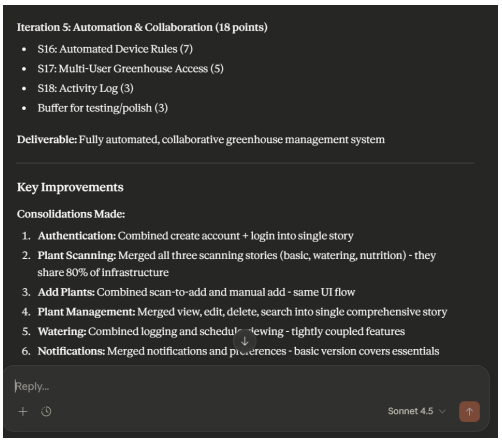
- S14: Connect & View Moisture Sensor Data (8)
- S7: View Greenhouse Dashboard (5)
- S15: Manual Device Control (5)

Note: Dashboard moved here since it needs device data to be meaningful

Deliverable: Full environmental monitoring with remote control

Iteration 5: Automation & Collaboration (18 points)

Reply...
Sonnet 4.5



Iteration 5: Automation & Collaboration (18 points)

- S16: Automated Device Rules (7)
- S17: Multi-User Greenhouse Access (5)
- S18: Activity Log (3)
- Buffer for testing/polish (3)

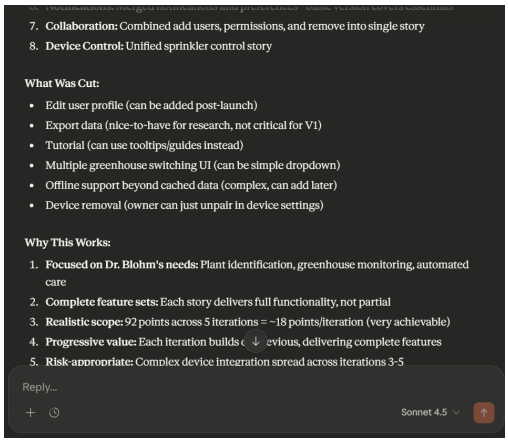
Deliverable: Fully automated, collaborative greenhouse management system

Key Improvements

Consolidations Made:

1. **Authentication:** Combined create account + login into single story
2. **Plant Scanning:** Merged all three scanning stories (basic, watering, nutrition) - they share 80% of infrastructure
3. **Add Plants:** Combined scan-to-add and manual add - same UI flow
4. **Plant Management:** Merged view, edit, delete, search into single comprehensive story
5. **Watering:** Combined logging and schedul dewing - tightly coupled features
6. **Notifications:** Merged notifications and pr erences - basic version covers essentials

Reply...
Sonnet 4.5



7. **Collaboration:** Combined add users, permissions, and remove into single story
8. **Device Control:** Unified sprinkler control story

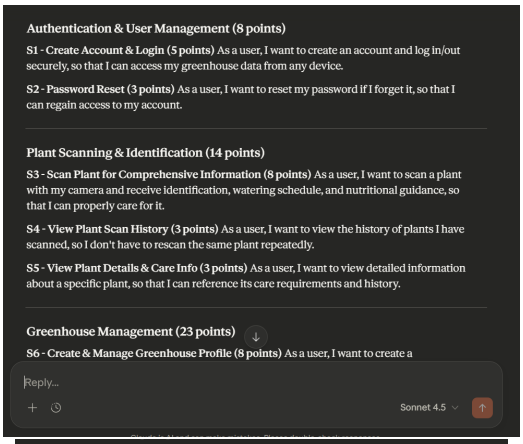
What Was Cut:

- Edit user profile (can be added post-launch)
- Export data (nice-to-have for research, not critical for V1)
- Tutorial (can use tooltips/guides instead)
- Multiple greenhouse switching UI (can be simple dropdown)
- Offline support beyond cached data (complex, can add later)
- Device removal (owner can just unpair in device settings)

Why This Works:

1. **Focused on Dr. Blohm's needs:** Plant identification, greenhouse monitoring, automated care
2. **Complete feature sets:** Each story delivers full functionality, not partial
3. **Realistic scope:** 92 points across 5 iterations = ~18 points/iteration (very achievable)
4. **Progressive value:** Each iteration builds evious, delivering complete features
5. **Risk-appropriate:** Complex device integration spread across iterations 3-5

Reply...
Sonnet 4.5



Authentication & User Management (8 points)

S1 - Create Account & Login (5 points) As a user, I want to create an account and log in/out securely, so that I can access my greenhouse data from any device.

S2 - Password Reset (3 points) As a user, I want to reset my password if I forget it, so that I can regain access to my account.

Plant Scanning & Identification (14 points)

S3 - Scan Plant for Comprehensive Information (8 points) As a user, I want to scan a plant with my camera and receive identification, watering schedule, and nutritional guidance, so that I can properly care for it.

S4 - View Plant Scan History (3 points) As a user, I want to view the history of plants I have scanned, so I don't have to rescan the same plant repeatedly.

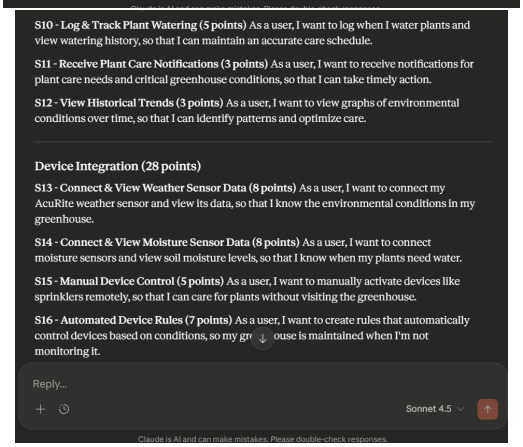
S5 - View Plant Details & Care Info (3 points) As a user, I want to view detailed information about a specific plant, so that I can reference its care requirements and history.

Greenhouse Management (23 points)

S6 - Create & Manage Greenhouse Profile (8 points) As a user, I want to create a

Reply...

Sonnet 4.5



S10 - Log & Track Plant Watering (5 points) As a user, I want to log when I water plants and view watering history, so that I can maintain an accurate care schedule.

S11 - Receive Plant Care Notifications (3 points) As a user, I want to receive notifications for plant care needs and critical greenhouse conditions, so that I can take timely action.

S12 - View Historical Trends (3 points) As a user, I want to view graphs of environmental conditions over time, so that I can identify patterns and optimize care.

Device Integration (28 points)

S13 - Connect & View Weather Sensor Data (8 points) As a user, I want to connect my AcuRite weather sensor and view its data, so that I know the environmental conditions in my greenhouse.

S14 - Connect & View Moisture Sensor Data (8 points) As a user, I want to connect moisture sensors and view soil moisture levels, so that I know when my plants need water.

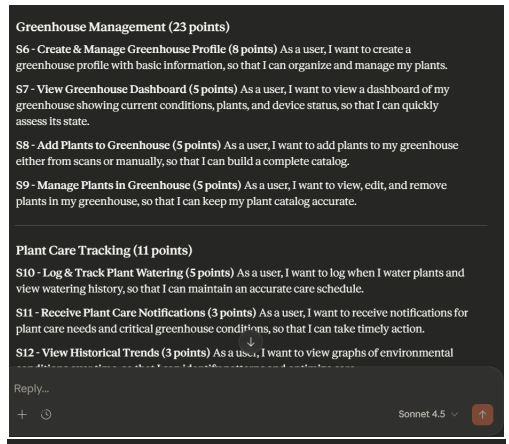
S15 - Manual Device Control (5 points) As a user, I want to manually activate devices like sprinklers remotely, so that I can care for plants without visiting the greenhouse.

S16 - Automated Device Rules (7 points) As a user, I want to create rules that automatically control devices based on conditions, so my gr ouse is maintained when I'm not monitoring it.

Reply...

Sonnet 4.5

Claude is AI and can make mistakes. Please double-check responses.



Greenhouse Management (23 points)

S6 - Create & Manage Greenhouse Profile (8 points) As a user, I want to create a greenhouse profile with basic information, so that I can organize and manage my plants.

S7 - View Greenhouse Dashboard (5 points) As a user, I want to view a dashboard of my greenhouse showing current conditions, plants, and device status, so that I can quickly assess its state.

S8 - Add Plants to Greenhouse (5 points) As a user, I want to add plants to my greenhouse either from scans or manually, so that I can build a complete catalog.

S9 - Manage Plants in Greenhouse (5 points) As a user, I want to view, edit, and remove plants in my greenhouse, so that I can keep my plant catalog accurate.

Plant Care Tracking (11 points)

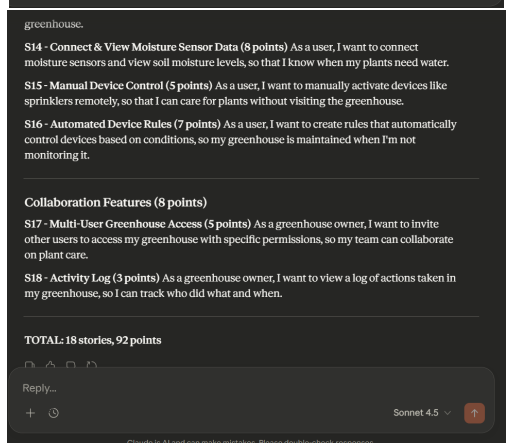
S10 - Log & Track Plant Watering (5 points) As a user, I want to log when I water plants and view watering history, so that I can maintain an accurate care schedule.

S11 - Receive Plant Care Notifications (3 points) As a user, I want to receive notifications for plant care needs and critical greenhouse conditions, so that I can take timely action.

S12 - View Historical Trends (3 points) As a user, I want to view graphs of environmental

Reply...

Sonnet 4.5



greenhouse.

S14 - Connect & View Moisture Sensor Data (8 points) As a user, I want to connect moisture sensors and view soil moisture levels, so that I know when my plants need water.

S15 - Manual Device Control (5 points) As a user, I want to manually activate devices like sprinklers remotely, so that I can care for plants without visiting the greenhouse.

S16 - Automated Device Rules (7 points) As a user, I want to create rules that automatically control devices based on conditions, so my greenhouse is maintained when I'm not monitoring it.

Collaboration Features (8 points)

S17 - Multi-User Greenhouse Access (5 points) As a greenhouse owner, I want to invite other users to access my greenhouse with specific permissions, so my team can collaborate on plant care.

S18 - Activity Log (3 points) As a greenhouse owner, I want to view a log of actions taken in my greenhouse, so I can track who did what and when.

TOTAL: 18 stories, 92 points

Reply...

Sonnet 4.5

4.2 Reflection

After asking A.I. to help find missing user stories, and reassess the story points, we now have a more detailed list of user stories with a total point tally of 78. A.I. was really helpful for coming up with more ideas, especially since we had a good foundation for it to work off of. There were some stories that it added that were unnecessary and inflated its point totals. There were also some stories it came up with that were more general and would need more specification to be a valid user story. We used some of those as a basis for some new stories. Something especially useful was the way the A.I. grouped the stories into different categories. This inspired us to revise our planning and work on things that were more related together.

Another interesting story was that of syncing the weather sensor data that would be pulled from an API. Dr. Isaacman's guidance was that this story could be overestimating the difficulty of the API calling, but A.I. found no issue with it. This reinforces the idea that A.I. should only be used as a guide. This was a valuable lesson on A.I. usage. The results we got certainly could not have been used immediately and had to be revised, and they were helpful because the A.I. had a lot of context to work with. This shows how A.I. can really shine; as a tool to assist a software engineer with a good idea and expand on it.

5 UI Mockups

The application will maintain visual consistency through a green and white color scheme with high contrast for outdoor readability.

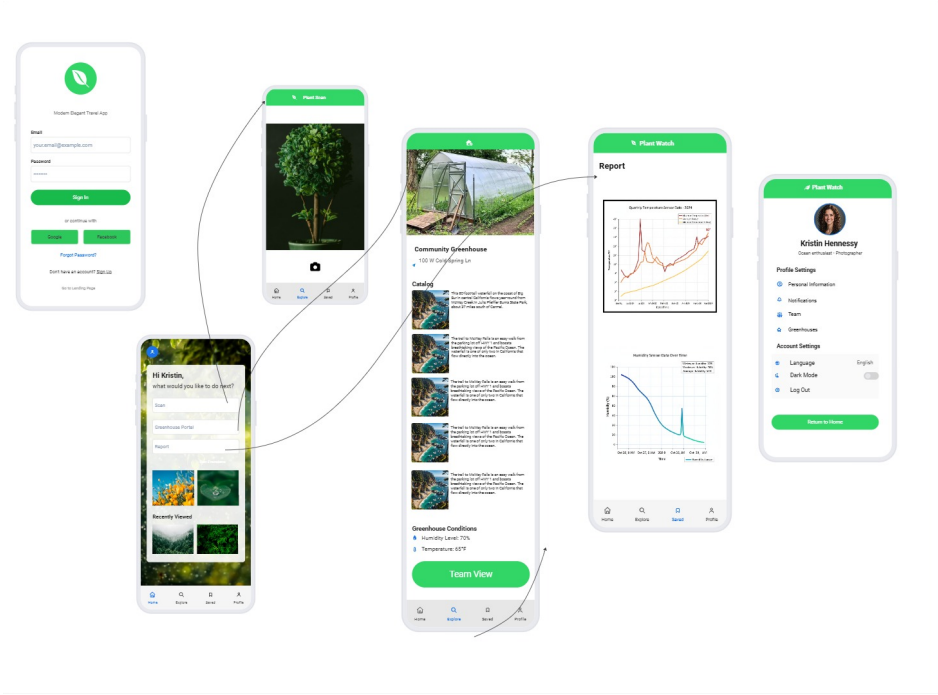


Figure 3: PlantWatch mobile application UI mockups

6 System Design

6.1 Architecture

We will be using a layered architecture. The layers we will have are:

- **Presentation:** Front-end User Interfaces
- **Application:** App logic and API calls
- **Data:** Database

We have chosen this architecture because we plan to use Supabase, a backend service that simplifies client server communications. Logic and processing will be done on the client when possible, and if the load is too strenuous or the server needs to do some processing, Supabase Edge Functions will be used. The server's primary role will be to hold data.
Figure 4 has a simplified class diagram that demonstrates the architecture. The list of front end pages is not exhaustive. The device page is intentionally one class and kept concise because we only know one specific model of device we want to be compatible with our app at the moment. We are still working with our client to decide on what other devices to integrate.

6.2 Diagrams

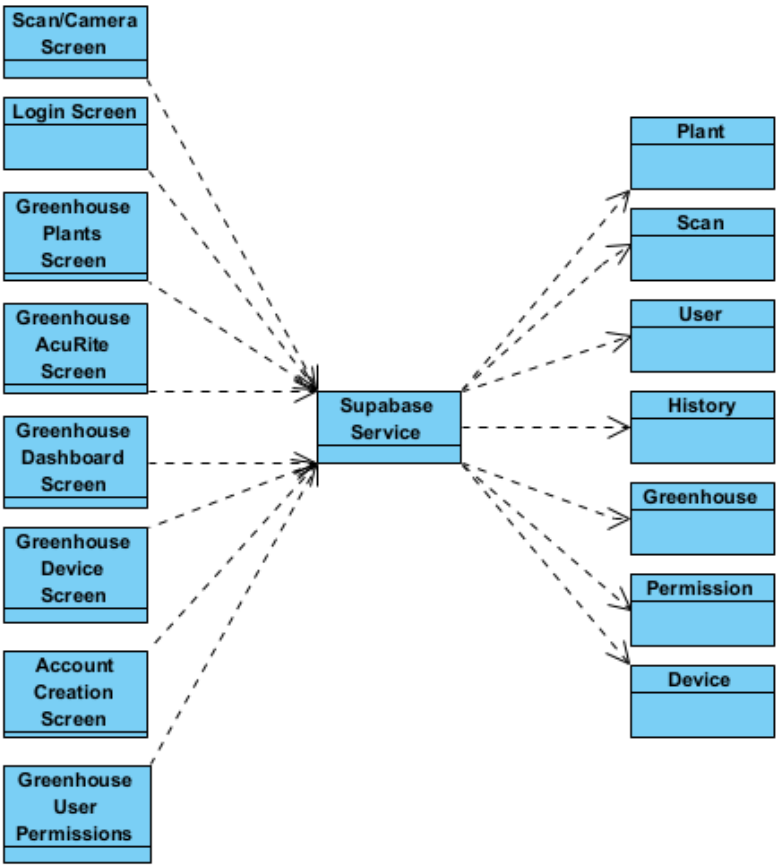


Figure 4: Simplified Class Diagram of App

6.3 Technology

Framework: React Native with Expo
Backend Services: Supabase
Language: JavaScript and Typescript
Primary Database: PostgreSQL 15+
Plant Identification: Pl@ntNet API
Weather Sensor: AcuRite → Weather Underground → Weather Underground API, or use RTL-433 workaround (AcuRite has no direct API)
Testing: Jest

6.4 Coding Standards

We will be following the coding standards below to maintain consistency and quality:

Naming Conventions

- **Database tables:** lowercase (e.g., `user`, `plant`, `greenhouse`)
- **Database Columns:** snake_case (e.g., `created_at`, `moisture_level`)
- **Methods/functions:** snake_case (e.g., `get_plant_info()`, `scan_plant()`)
- **Classes:** PascalCase (e.g., `PlantScanner`, `GreenhouseManager`)
- **Constants:** UPPER_SNAKE_CASE (e.g., `MAX_SCAN_TIMEOUT`)

Testing Requirements

- Minimum 70% code testing coverage for all commits

Code Review Process

- We will review and approve all code changes from our independent branches before they are added to the main project
- The commit messages will include the story number and brief description: `[S3] Added plant scanning`

Quality & Security

- Sensitive information like passwords, API keys will be kept private using environmental variables.
- Encrypt all user passwords before storing them in the database to ensure user passwords are protected.
- Use parameterized database queries to prevent SQL injection attacks where malicious users attempt to manipulate the database
- Include error handling for all external connections like API, database, sensor, so the app doesn't crash when something goes wrong
- Attempt to not repeat codes and instead create reusable functions, so that changes only need to happen in one place, reducing redundancy and potential bugs.

Documentation

- Functions will have comments at the top explaining what it does, explaining the purpose, input, parameters and return values.
- Complex codes will include comments explaining what it does.
- We will document all API endpoints with examples of how to use them.

6.5 Data

Figure 4 contains a basic logical model of how we plan to structure data. So the diagram is not cluttered, some proposed attributes for the tables and relationships will be listed below the diagram. More are likely to be added when development begins.

Scan

- scan_dt
- scan_photo
- scan_description

User

- user_name
- password
- email

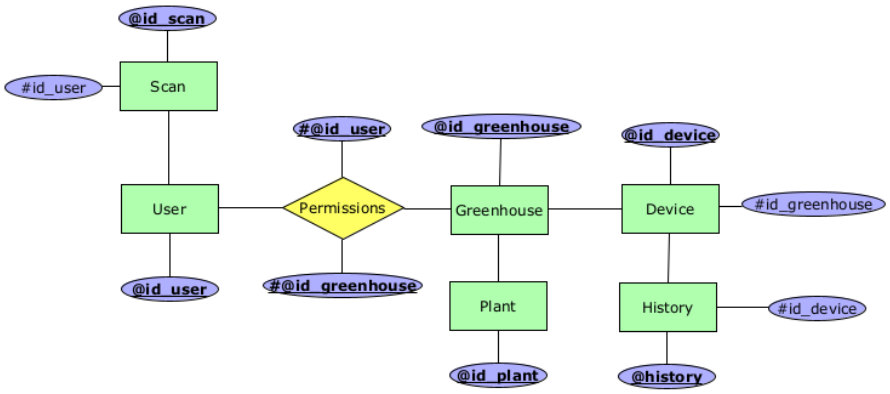


Figure 5: Basic Logical Model of Data

Greenhouse

- greenhouse_name

Permissions

- plant_edit
- device_use

Device

- device_name
- device_api

History

- history_details

Plant

- plant_name
- plant_description
- plant_count

7 Iterations

7.1 Iteration Planning

Table 3 shows the iteration planning.

			Points	
It.	**Dates**	**Stories**	**Planned**	**Done**
1	01/01 - 02/01	S01, S05, S04, S19	15	13
2	02/01 - 03/01	S16, S06, S09	18	9
3	03/01 - 04/01	S03, S07, S08, S17	16	6
4	04/01 - 05/01	S10, S12, S18	15	4
5	05/01 - 06/01	S2, S11, S13, S14, S15	12	–
		Total:	74	32

Table 3: Iteration Planning for Incremental Deliveries

7.2 Iteration/Sprint 1

7.2.1 Planning

The plan for this iteration was 4 stories that addressed account credentials and plant scanning (S1, S4, S5, S19) for a total of 15 points. These features are core to the apps functionality and allows us to interact with many parts of the frameworks and services we are using.

7.2.2 Work Done

This iteration was a slow start. Most of S1 for a total of 5 points in iteration 1. Plants can be scanned and basic identifying information will be displayed to the user about what plant it is. The setup of the app was a large learning process, we do not have mobile development experience and have not used React, Expo, or Supabase before. Researching these different tools and going through examples took up lots of time in this iteration. We are still trying to figure out authenticating through Supabase as well, that has not been straightforward. I (Brenden) was the only one who completed work this iteration, Nishant is out of the country and I have not received word from him.

7.2.3 Testing Coverage

Figure 6 shows the testing coverage for iteration 1. Part of learning how to use Supabase was learning how to integrate tests with it. We only ended up creating one server-sided function and that function does not have a test currently, so we have no testing coverage. This coverage is not good enough, but is not as bad as it seems. Most of the work done in this iteration has been front end work and backend setup through the Supabase portal. The one function that needs to be tested is an API call and some basic text reformatting. The coverage report attached shows several front end files, we have not completely figured out configuring the test suite to pick up relevant files. Coverage will be increased quickly as the suite is correctly configured and the test for the function is implemented. Future commits will include tests since the testing will be set up.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	0	0	0	0	
PlantWatch	0	0	0	0	
expo-env.d.ts	0	0	0	0	
PlantWatch/.expo/types	0	0	0	0	
router.d.ts	0	0	0	0	
PlantWatch/constants	0	100	100	0	
theme.ts	0	100	100	0	6-30
PlantWatch/hooks	0	0	0	0	
scanPlant.ts	0	0	0	0	1-24
use-color-scheme.ts	0	100	0	0	1
use-color-scheme.web.ts	0	0	0	0	1-20
use-theme-color.ts	0	0	0	0	6-19
PlantWatch/tests/utils	0	0	100	0	
supabase-client.ts	0	0	100	0	1-7

```
Test Suites: 1 failed, 1 total
Tests:       0 total
Snapshots:   0 total
Time:        7.047 s
Ran all test suites matching ./tests.
```

Figure 6: Iteration 1 test coverage report

7.2.4 Retrospective & Reflection

Brenden
The major challenge I had in this iteration was the unfamiliarity I had with the tools we decided to use. Supabase is a backend replacement and takes a good amount of reading/learning to understand. I have not worked with mobile before, and learning how to use React Native and Expo was also a long experience. I've gotten more comfortable with all the components of the project, but I did not give myself enough time to apply the concepts. Next iteration, I will begin working right away and steadily make progress towards completing stories. I also found that watching videos on how other set up their Supabase projects has been helpful, so I will do that prior to jumping into coding more often especially in this second iteration. Another thing I fell short on was commenting, so I will make sure to do that along the way this time as well. It will especially be helpful for understanding what I am doing, and cluing my partner into what I have done. I have learned way too much to detail here this iteration, so I will just summarize it as basics of React Native, Expo, and Supabase and usage of API keys.

7.3 Iteration/Sprint 2

7.3.1 Planning

For Iteration 2, we planned the following user stories:

Story	Description	Points
S6	Greenhouse Profile Management	8
S9	Add Plants to Greenhouse	8
S16	Greenhouse Plant Info	2
Total		**18**

The planned focus of Iteration 2 was establishing the core greenhouse management experience. S6 and S9 are closely related. Before a user can add plants, they need to be able to create and manage a greenhouse profile. S16 builds on this by displaying plant and sensor information on a greenhouse dashboard. The goal of this iteration was to move beyond basic authentication and scanning, and provide the user with the greenhouse-centric features that are central to the enhancing the overall user experience.

7.3.2 Work Done

Story	Description	Status	Assigned To
S5	Create Account	3/3	Brenden
S19	Login/Logout	1/1	Brenden
S4	View Plan Scan History	1/3	Brenden
S6	Greenhouse Profile Management	3/8	Nishant
S9	Add Plants to Greenhouse	3/8	Nishant
S7	Greenhouse Profile Dashboard	1/3	Nishant

Brenden
S5, S19, and S4 were carried over from Iteration 1. Accounts are now properly set up, and users can create accounts, login from them, and logout. The plant scan history did not quite get finished, the backend database logging is implemented but the front end needs to be created, that is being done in the beginning over Iteration 3.

Nishant
S6 was completed by building a greenhouse management screen where users can create, view, switch between, and delete greenhouses.

S9 was completed by building a plants screen that is linked to a selected greenhouse. When adding a plant to a specific greenhouse, the app ensures that the `greenhouse_id` is correctly associated with that greenhouse in the Supabase `plants` table. Users now have the ability to add a plant with a common name, scientific name, and notes, and can also delete plants from the list.

S16: Pushed to later iteration.

S7 was started, app shows temperature, humidity, and notification section that are hardcoded.

7.3.3 Testing Coverage

Brenden
This iteration still does not have testing fully set up, so no coverage report is included as it would be the same as the last iteration.

Nishant
No formal test coverage was built for Iteration 2. The focus was on building a functioning frontend (Greenhouse Portal) and connecting it with Supabase for a seamless data implementation for future iterations. This included designing and implementing the greenhouse management screen, the plants screen, and the greenhouse dashboard UI, as well as setting up the database tables for greenhouses and plants in Supabase. Given the frontend-heavy nature of this iteration — navigating dependency conflicts (package.json had mismatched versions of React), setting up the Supabase connection, and a user-centric UI design — I relied heavily on manual testing through the Expo Go application on a physical device; verifying manually by creating, viewing, switching, and deleting greenhouses, as well as adding and removing plants, and confirming that all changes were correctly reflected in the Supabase dashboard. To ease this load for future iterations, the plan is to write unit tests for the core Supabase functions such as `fetchGreenhouses`, `addPlant`, and `deleteGreenhouse`.

7.3.4 Retrospective & Reflection

Brenden
I wasn't as productive as I had hoped this iteration, but thankfully the authentication was set up. Getting over that hurdle is good though, now auth can be used to secure other features that rely on authentication. The experience helped me understand the setup of the project. Testing definitely needs to be set up, I still had a poor understanding of what that looks like considering the heavy front-end and use of supabase authentication functions. Planning to ask for guidance on this.

Nishant
One of the initial challenges of Iteration 2 was resolving dependency conflicts in the project. After reading through a few threads on StackExchange, I found out that the app was failing because the `package.json` had mismatched versions of React, React Native, and Expo. To resolve this, I downgraded React Native to `0.74.5` and React to `18.2.0` to match Expo SDK 51, and running `npm install --legacy-peer-deps` to bypass remaining conflicts. Along with that, establishing the Supabase connection was also challenging. Initially, no requests from the app were being passed to the database. I then found out that it was because Row Level Security (RLS) was enabled by default with no policies written. Furthermore, I relied too heavily on manual testing, which was time-consuming and could be improved in that sense by creating automated testing for a smoother and more thorough functioning of the app.

7.4 Iteration/Sprint 3

7.4.1 Planning

For Iteration 3, we planned the following user stories:

Story	Description	Points
S3	Add Plants to Greenhouse by Scan	3
S7	Greenhouse Profile Dashboard	3
S8	Sync AcuRite Weather Sensor	5
S17	Sync Soil Moisture Sensor	5
Total		**16**

The main focus for Iteration 3 is hardware integration and delivering a more complete greenhouse dashboard experience for the users.

7.4.2 Work Done

Story	Description	Status	Assigned To
S3	Add Plants to Greenhouse by Scan	3/3	Nishant
S7	Greenhouse Profile Dashboard	3/3	Nishant
S8	Sync AcuRite Weather Sensor	0/5	Brenden
S17	Sync Soil Moisture Sensor	0/5	—

Brenden

Spent some time this iteration reworking the app to use Expo's routing after seeing Nishant used it in some of his work on the Greenhouse pages. I also worked with S6, making it so that users are connected to greenhouses in the database through a permissions table instead of being public. I extracted the functionality of some of the methods that were in the front-end React and created a greenhouse service. Testing has been a pain point for us, so I spent some time fixing up the testing framework and created some tests for the greenhouse functions.

Nishant

S3: Added the feature to submit and add plants directly to a greenhouse by scanning.

S7: Updated the pre-existing Greenhouse Dashboard from Iteration 2. The improved dashboard displays a list of greenhouses. When the user clicks on one of the greenhouse, it shows the temperature, humidity, last updated timestamp, and a list of all plants belonging to that greenhouse. Sensor data is currently displayed as mock values, pending hardware integration in S8 and S17.

S8 and S17 were not completed in this iteration. Connecting the AcuRite Weather Sensor and Soil Moisture Sensor requires additional hardware setup which will be carried over to the next iteration.

7.4.3 Testing Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	29.67	38.46	38.46	27.27	
auth.ts	0	100	0	0	1-33
greenhouse.ts	100	100	100	100	
plant.ts	0	0	0	0	2-132

Figure 7: Iteration 3 test coverage report

Brenden

I finally set up the testing environment properly this iteration. I first started with an integration test that tested the functionality of the server side function that reaches out to PlantNet API, and I set up some tests unit tests for some of the greenhouse functions. The coverage report shows all of the client services that need to be tested. I have not yet found a front-end framework for testing the React easily, will update on that next iteration.

Nishant

I relied heavily on manual testing for Iteration 3 to ensure the front-end design was working well, and so I have no automated testing to report on. Having said that, certain parts of the app are difficult to test on as it relies on external hardware that communicates directly with the phone.

7.4.4 Retrospective & Reflection

Nishant: The biggest challenge in this iteration was hardware integration. It is a difficult user story to complete and will require more groundwork, specifically establishing a reliable data pipeline from the physical

devices to Supabase. To address this, we need to research deeply into the hardware setup and allocate more time specifically for sensor configuration and testing.

On the database side, I had deliberately disabled Row Level Security (RLS) on most Supabase tables during Iteration 2 as a quick fix to not halt the development of the app. While this allowed me to build on the app, it also means that any authenticated user can currently read and modify all data in the database. This is a known limitation and will be addressed along with the hardware integration into our app.

This iteration, although difficult has reinforced the importance of building and breaking the app time and time again to improve my understanding. A small oversight in how a `greenhouse_id` is passed between screens can break an entire feature. Furthermore, I've learnt of potential ways of integrating hardware into the app, specifically how a microcontroller like a Raspberry Pi can read sensor data and push it to Supabase. This will give us a clearer technical direction for completing S8 and S17 in the next iteration. Iteration 3 has come with its challenges, and at times, the development environment proved demanding. But this learning experience has deepened my understanding of how frontend and backend systems interact in a real mobile application.

Brenden

Nishant covered most of the hardware challenges so I won't belabor the point, but this iteration hardware was difficult to work with. Turns out the weather sensor we're working with doesn't have any connection to the internet, so we'd have to get an external device to pick up the radio signals. Doing this would be a cool experience and I would be open to it, but going to have to discuss with our client about how we want to move forward. The soil sensor has arrived, just haven't tested it out yet. Instead I worked on what Nishant mentioned about the databases. I've added RLS policies for the greenhouse and connected greenhouse to users (before all greenhouses were shared). It was surprisingly annoying to get the RLS policies just right, took lots of experimenting and testing to figure out.

7.5 Iteration/Sprint 4

7.5.1 Planning

For Iteration 4, we planned the following user stories:

Story	Description	Points
S10	Greenhouse Notifications	5
~~S12~~	~~Scheduled Sprinkler Activation~~	~~5~~
S18	View Historical Conditions	5
Total		**15**

7.5.2 Work Done

Story	Description	Status	Assigned To
S10	Greenhouse Notifications	3/3	Nishant
~~S12~~	~~Scheduled Sprinkler Activation~~	~~0/5~~	–
S18	View Historical Conditions	1/5	Brenden
S17	Sync Soil Moisture Sensor	0/5	Brenden
S9	Add Plants to Greenhouse	$3/8 \rightarrow 6/8$	Brenden
S16	Greenhouse Plant Info	2/2	Brenden

Brenden:

Unfortunately due to hardware complications I could not set up the soil moisture sensor, more elaboration on that is in the reflection. Our client did not purchase a sprinkler, so that story will not be completed. Since we have not connected any hardware to the app yet, there are no historical conditions to view at this time. I've started some setup to support it in the backend, but it is hard to know how to structure things without any hardware successfully integrated. So this iteration, most of my time spent was on research and improving existing features, like expanding on our plant creation and improving the greenhouse page.

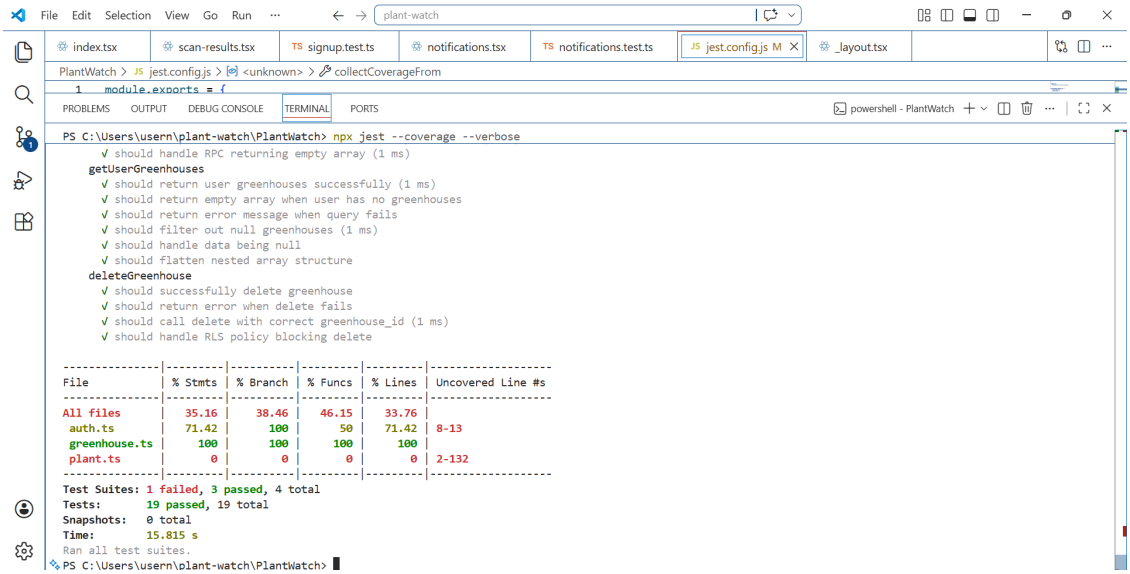


Figure 8: Testing Coverage

7.5.3 Testing Coverage

The current testing reports has low coverage. The report is 35.16% for statements, 38.46% for branches, 46.15% for functions, and 33.76% for lines. To improve on this, we need to write a functioning and a more detailed test for plants.ts, fix the failing suites, and add more tests for the app. By writing meaningful unit tests, that focuses on testing all aspects,and possible scenarios of each function.

7.5.4 Retroespective & Reflection

Brenden:
This iteration came with more issues with hardware. I picked up the soil sensor and this time it has an app associated with it, but it is a closed network app with no public facing API. If I were to get information off this app somehow, I'd have to packet sniff while the sensor is in action, intercept the API, and replicate its structure in our program. This isn't a very secure/reliable workaround because the device this time around is connected to internet. This means it can receive software updates and its API call structure is vulnerable to change. This roundabout method also might be violating their TOS, since going closed network was a deliberate and recent change. We'll have to discuss with our client about this difficulty. This experience taught us that in a project that involves external hardware like this, the selection of the hardware should be more of a collaborative process. We should have worked with our client to find sensors that have public facing APIs and suit our clients needs. Besides hardware, I was not able to find a suitable automated testing framework for front end tests, so we will be using the react-native Jest library to test our front-end components in this last iteration. Lastly, we have not been communicating with each other the best these last few iterations. We going to ramp up our cooperation and development in this last stretch to get as many features as we can done, despite the complications with hardware and whatnot.

Nishant: Like Iteration 3, Iteration 4 has still kept us in this loop of trying to find a working solution for integrating hardware remotely. In addition, S12 was removed from this iteration as there are no sprinklers in the greenhouse.

For this iteration, I worked on getting the notification system set up, along with updating the test suite which only tested files in the path `'services/**/*.ts,tsx'`. In addition to that, I found a concrete solution to integrating hardware into our code, which will require additional materials but a solution is in sight. It is through the use of a microcontroller, specifically ESP32-C6 connected to a SHT-C3 (a temperature and humidity sensor) which through MicroPython will help in integrating hardware into our app.

Finally, I looked into maximizing Battery Efficiency, **NFR7**, as per Dr. Isaacman's comments in the

previous document. I came to know that performance in React Native apps can be optimized by using a FlatList or a SectionList when rendering large lists. And about functions, such as useMemo and useCallback that avoids unnecessary calculations and saves space, and memory. After letting the app run for about 20 minutes, the CPU performance read between 256 MB to 356 MB, which after some research, I understood that it was negligible and kept the Battery Consumption below 5% of the device, which was our goal. Having said that, our app is currently a skeletal picture of what the full app will be. More data in the database, a more robust and accurate camera system, a fully functioning remote monitoring dashboard, among other things. So, implementing functions such as useMemo and UI component such as a FlatList will be crucial in maintaining the Battery Efficiency standard. Lastly, Communication has been the biggest pain point during this iteration. We plan on keeping each other updated as we work on Iteration 5.

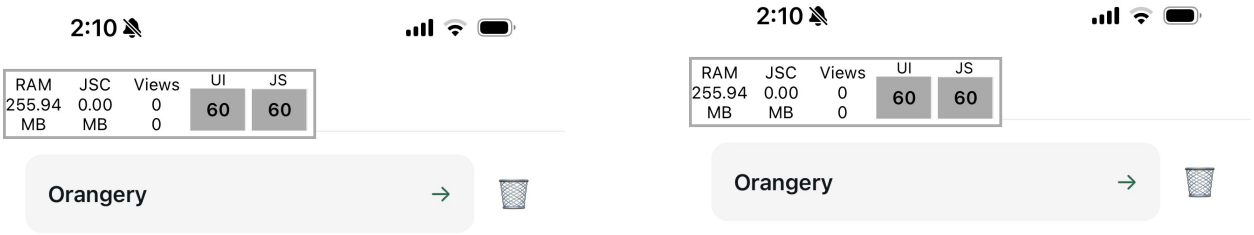


Figure 9: CPU Performance

7.6 Iteration/Sprint 5

7.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

7.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

7.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

7.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

8 Final Remarks

8.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

8.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]